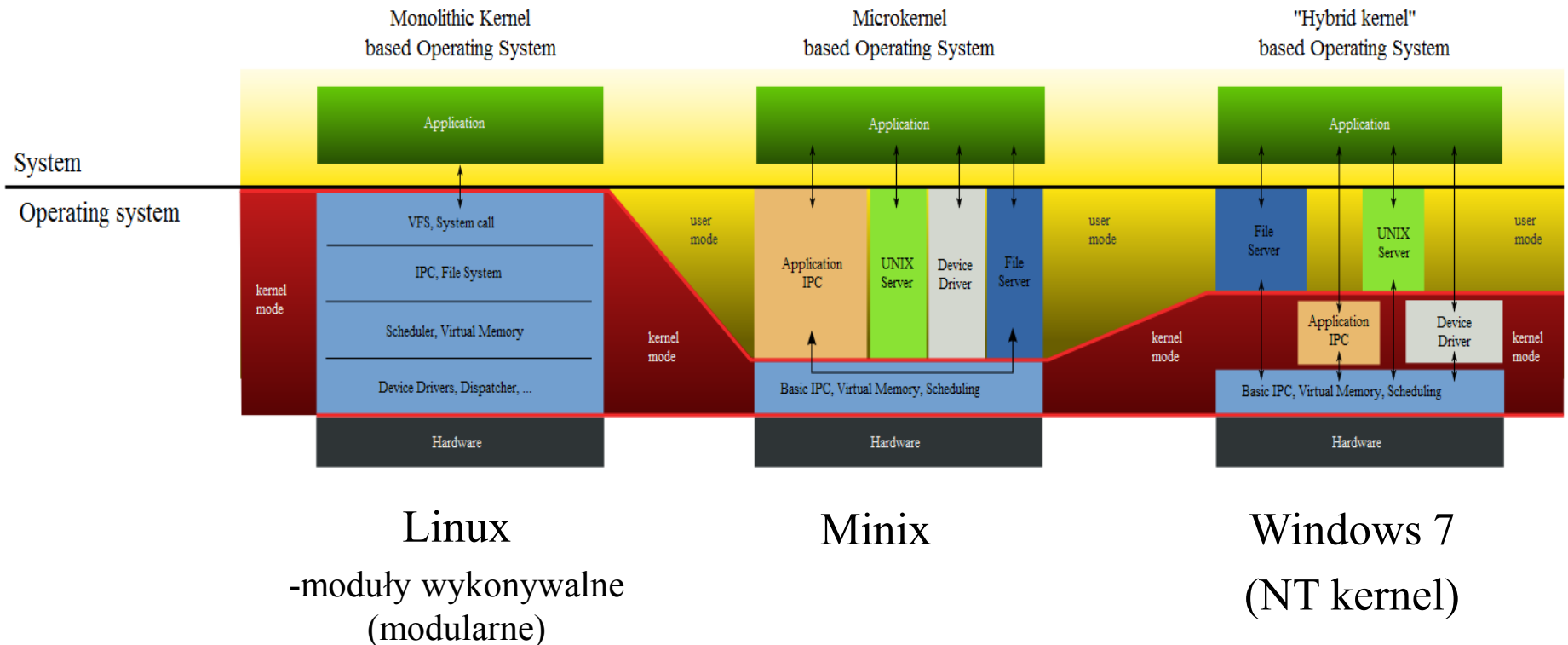
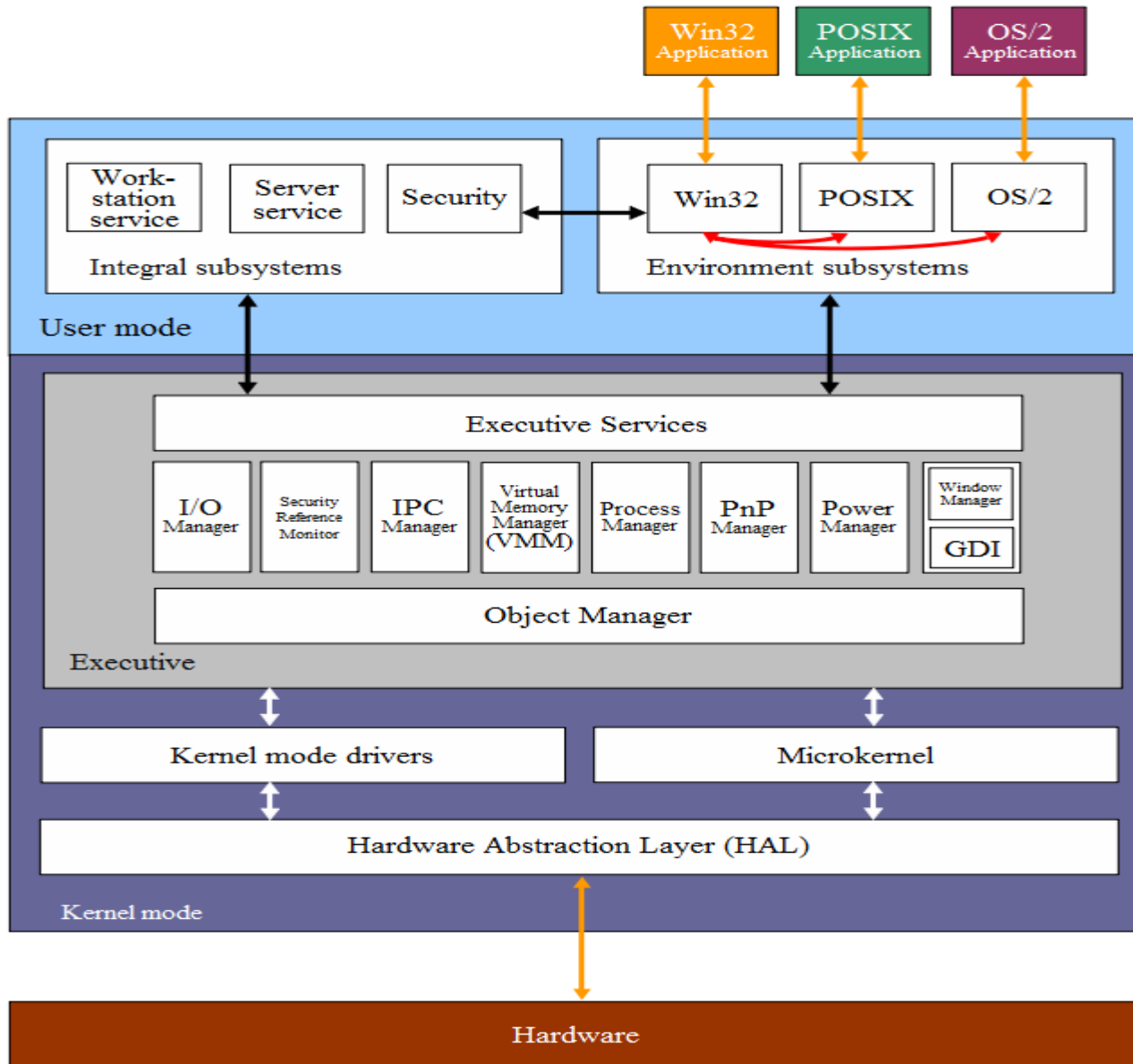


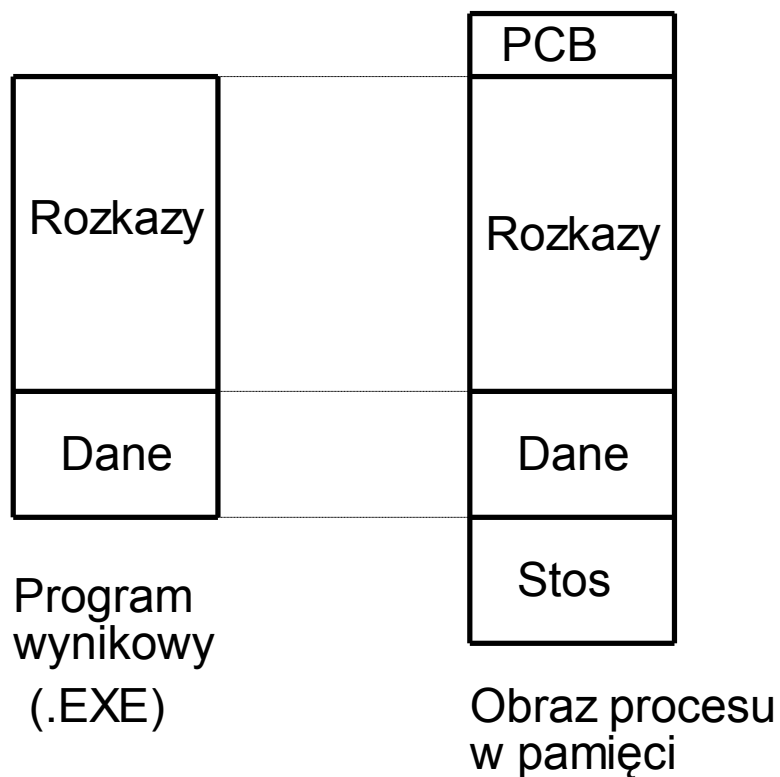
Podstawowe architektury SO



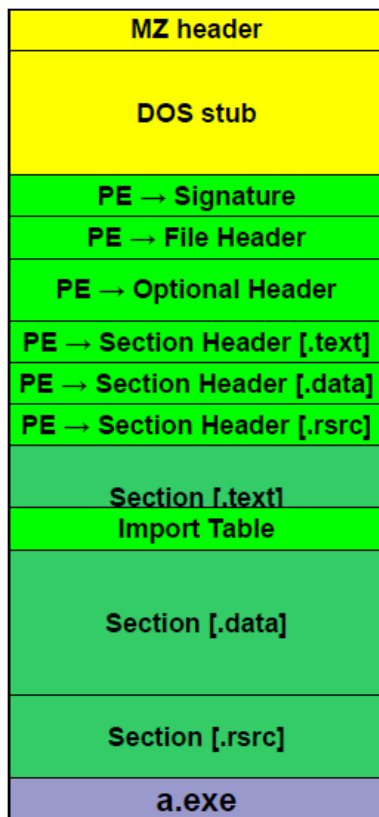
Architektura Windows



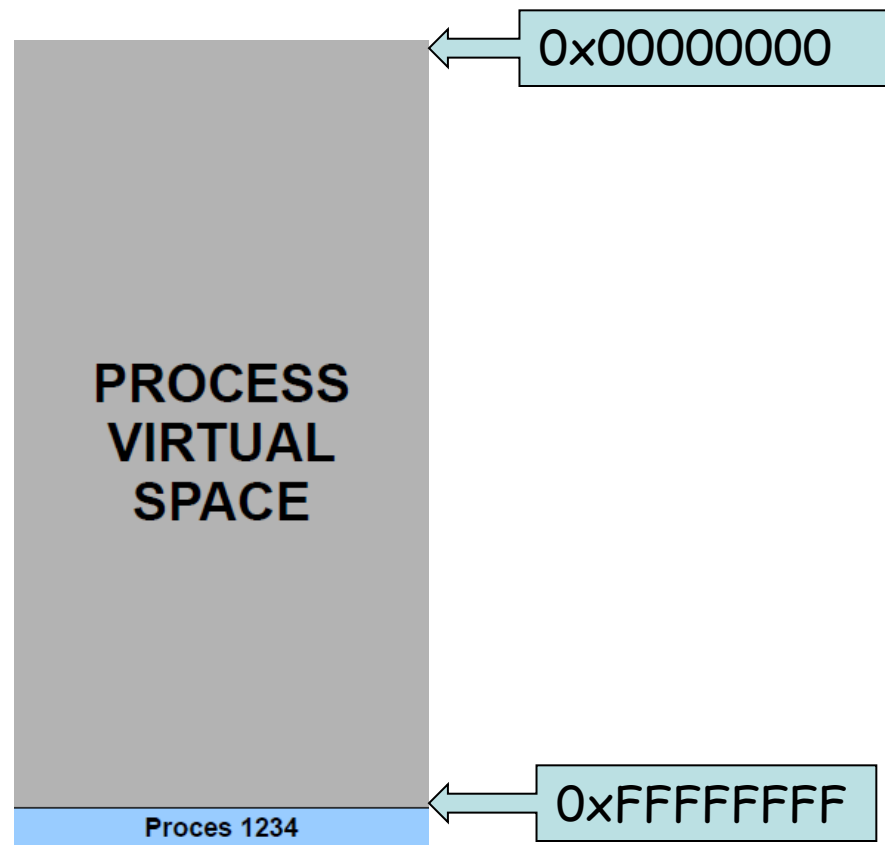
Ładowanie programów do pamięci - > proces



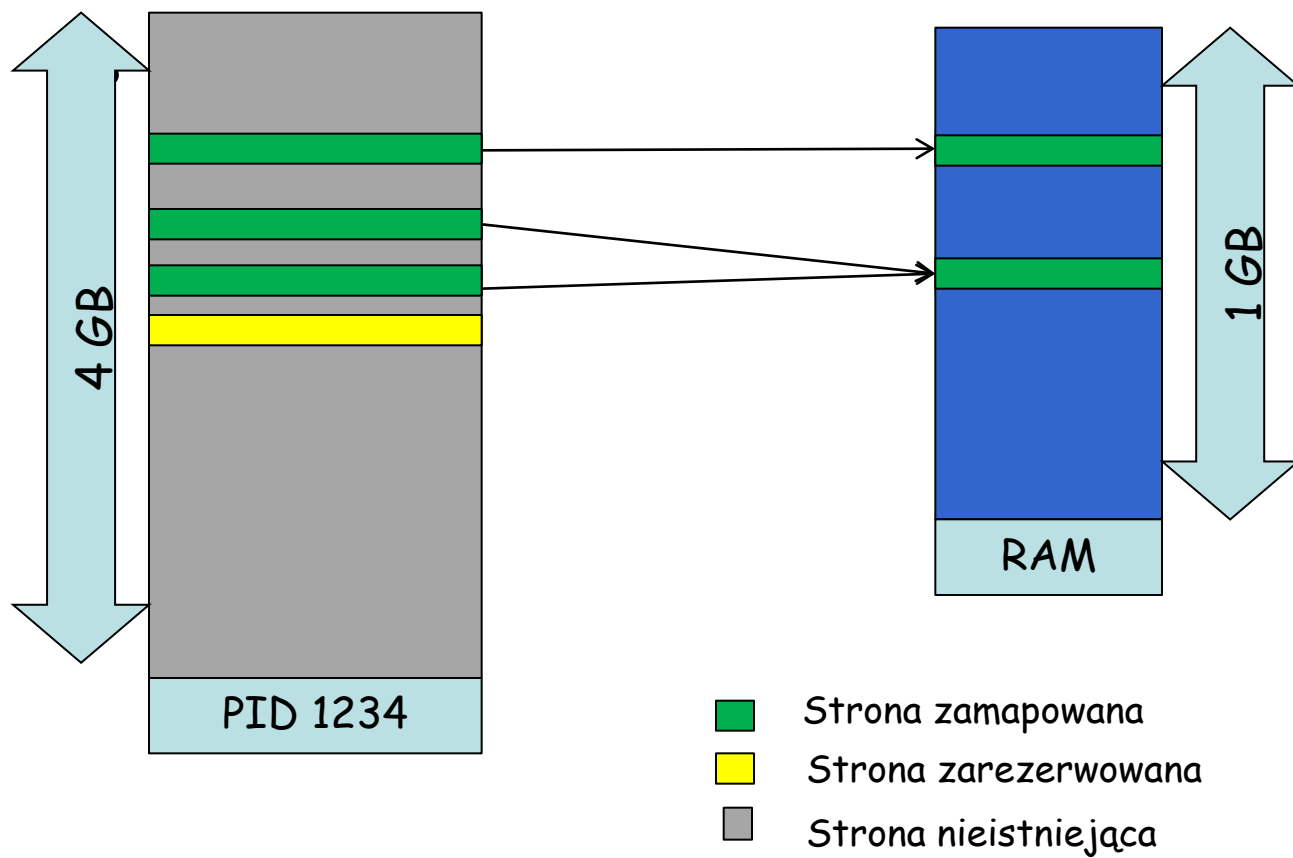
Proces to kontener z zasobami używanymi podczas wykonywania instancji programu

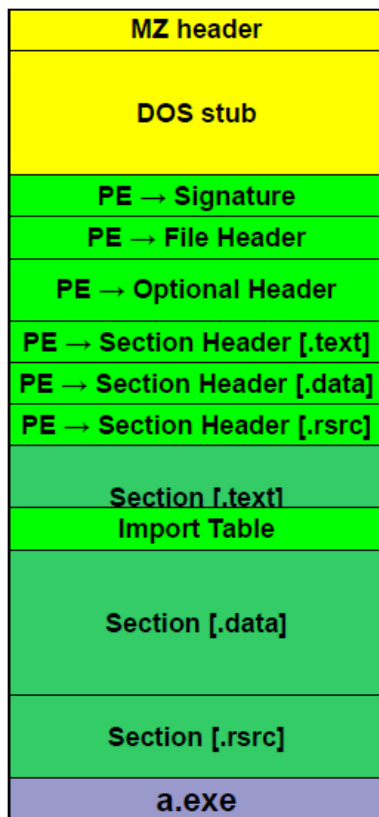


Plik

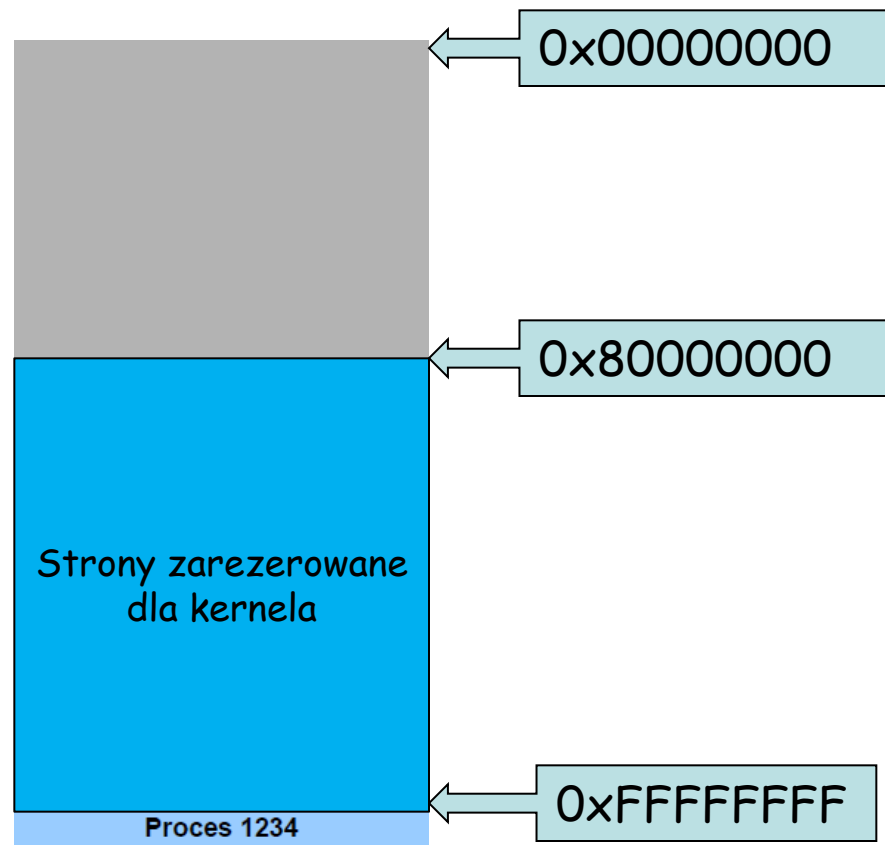


Pamięć wirtualna
przydzielona procesowi

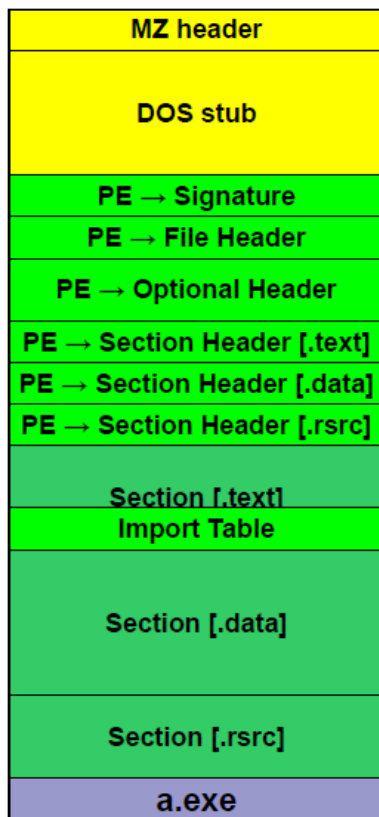




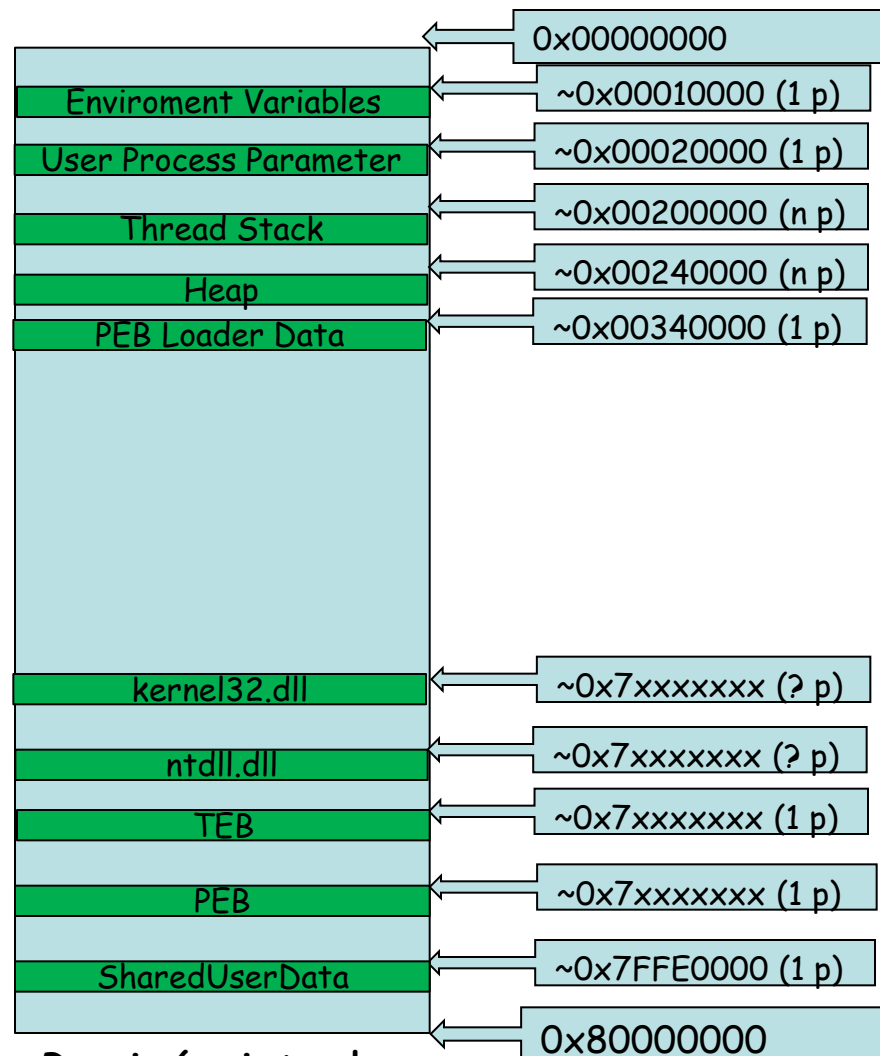
Plik



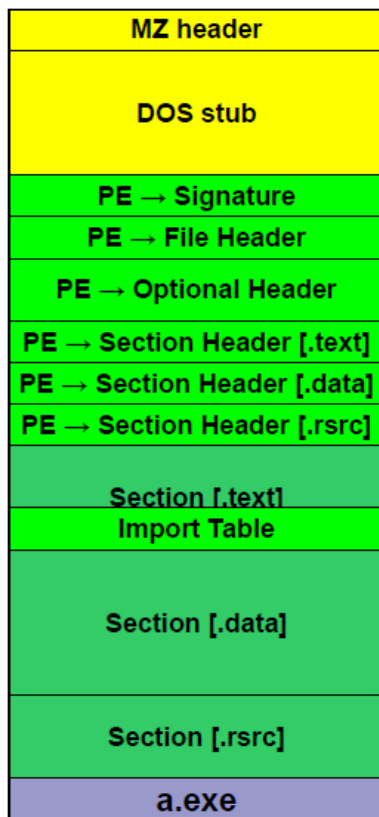
Pamięć wirtualna
przydzielona procesowi



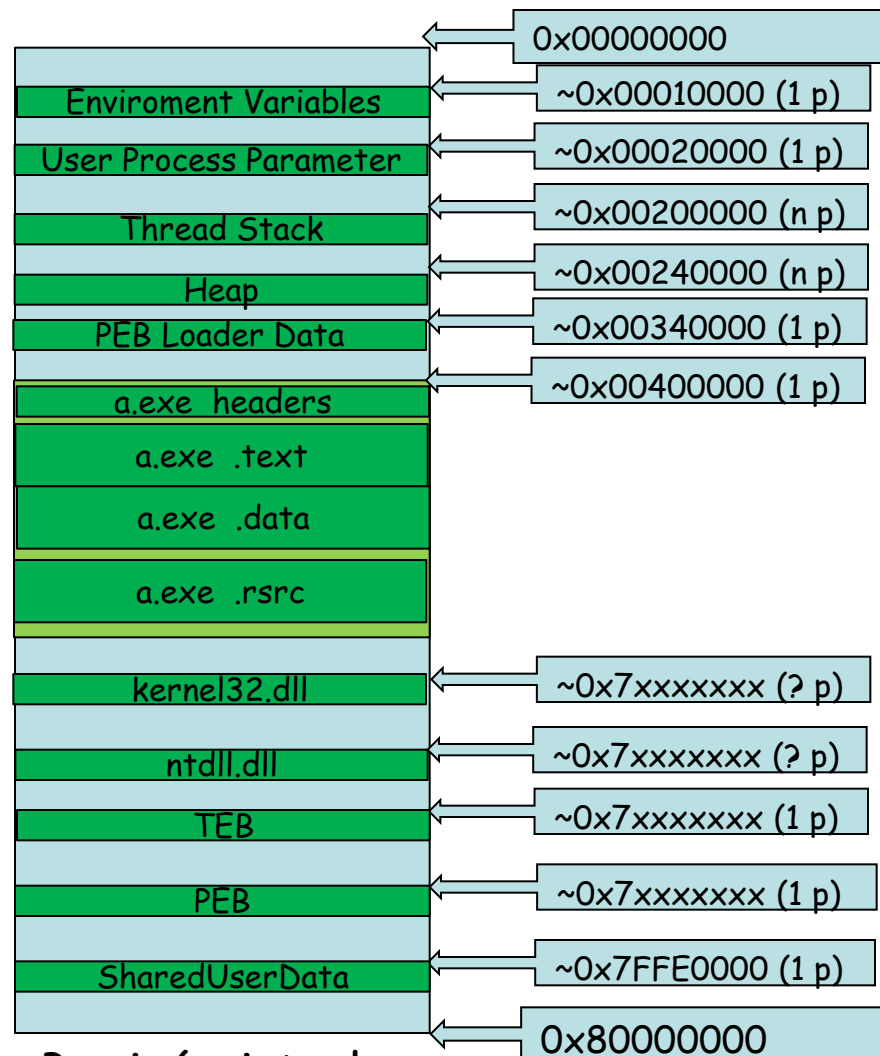
Plik



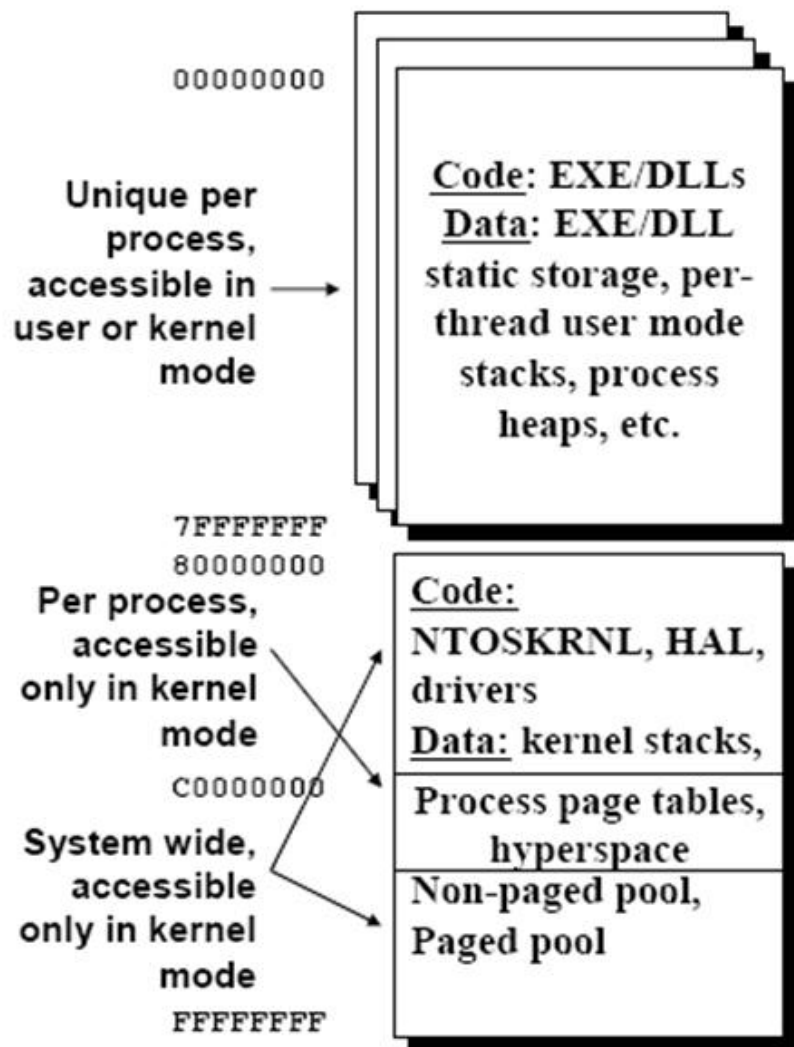
Pamięć wirtualna
przydzielona procesowi



Plik



Pamięć wirtualna
przydzielona procesowi



- **2 GB per-process**
 - Address space of one process is not directly reachable from other processes
- **2 GB system-wide**
 - The operating system is loaded here, and appears in every process's address space
 - The operating system is not a process (though there are processes that do things for the OS, more or less in "background")
- **3 GB user space and Address Windowing Extensions (AWE) t.b.d.**

Struktury systemowe procesu w Windows

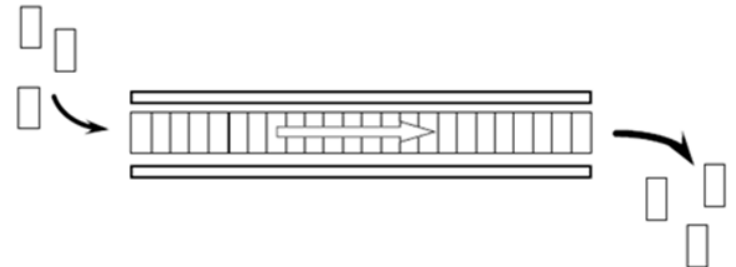
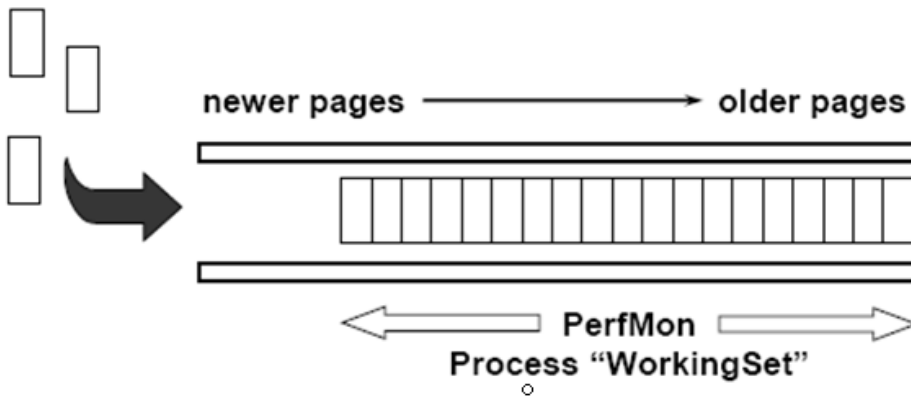
- Struktury danych
- Zmienne systemowe
- Mierniki wydajności
- Funkcje API
- Przepływ sterowania przy tworzeniu procesu

Prywatnej wirtualnej przestrzeni adresowej - zbiór adresów pamięci wirtualnej do dyspozycji procesu.

- Programu wykonywalnego - zmapowany w pamięci kod i dane początkowe.
- Listy wskaźników itp. do różnych zasobów takich jak semaforey, porty komunikacyjne, pliki, dostępne dla wszystkich wątków w procesie.
- Znacznika dostępu - struktura zawierająca security ID, listę grup, jakich członkiem jest dany użytkownik, przywileje. Znacznik dostępu jest dziedziczony po procesie-rodzicu.
- Unikatowego identyfikatora procesu.
- Przynajmniej jednego wątku wykonawczego.
- Wskaźnika do procesu nadrzędnego

Struktury danych

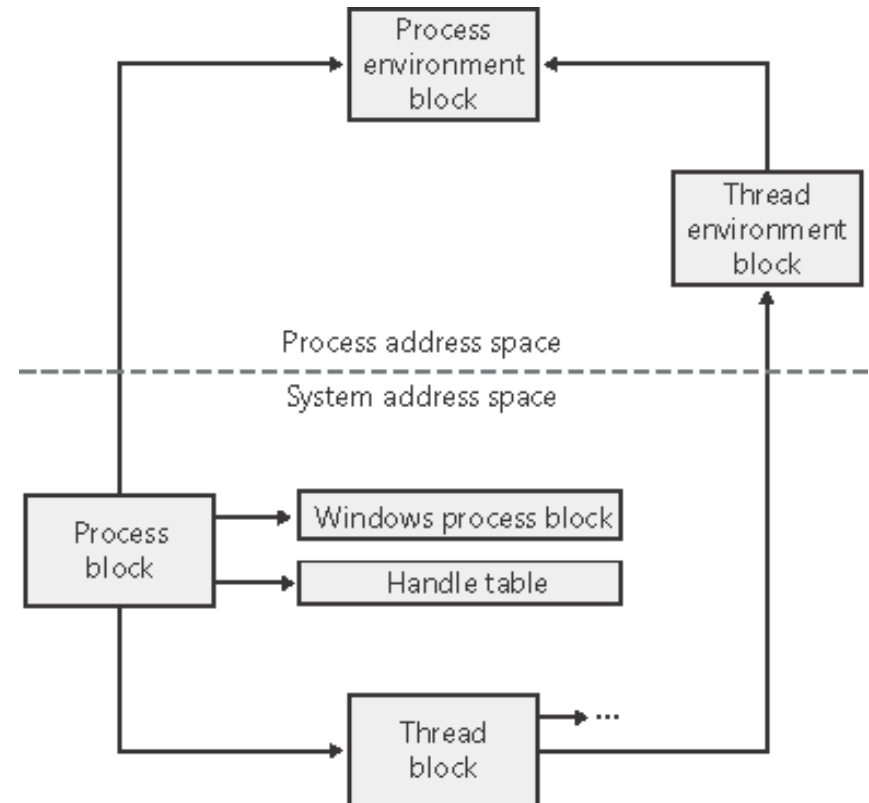
- Virtual Address Descriptors (VADs)
 - Zbiór struktur do śledzenia który z adresów wirtualnych w przestrzeni adresowej procesu jest zarezerwowany a który nie.
 - Opisuje stan przestrzeni adresowej procesu
- Working Set
 - Working set jest podzbiorem stron, które rezydują w pamięci fizycznej.



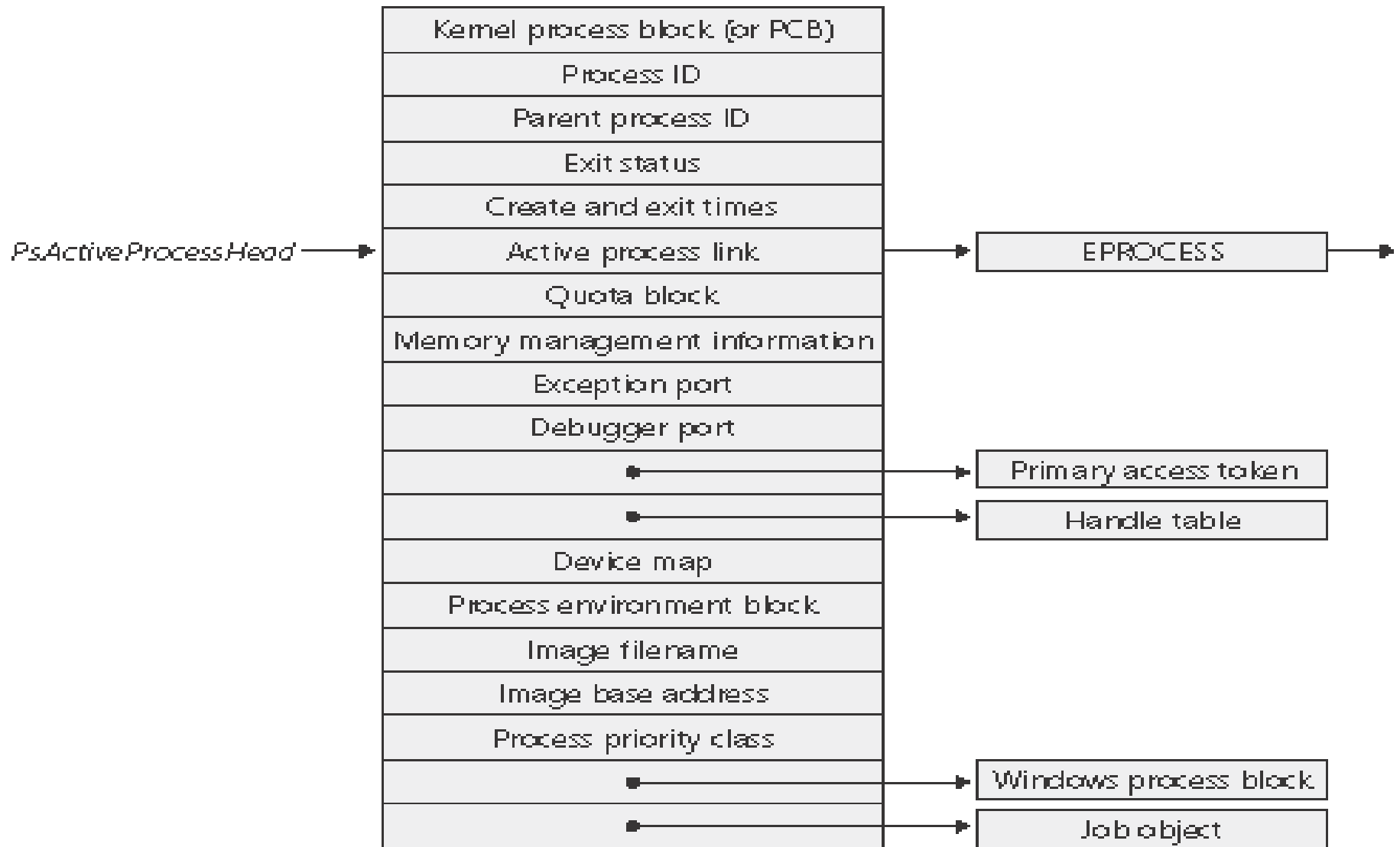
Struktury systemowe procesu w Windows

Z procesem powiazane są

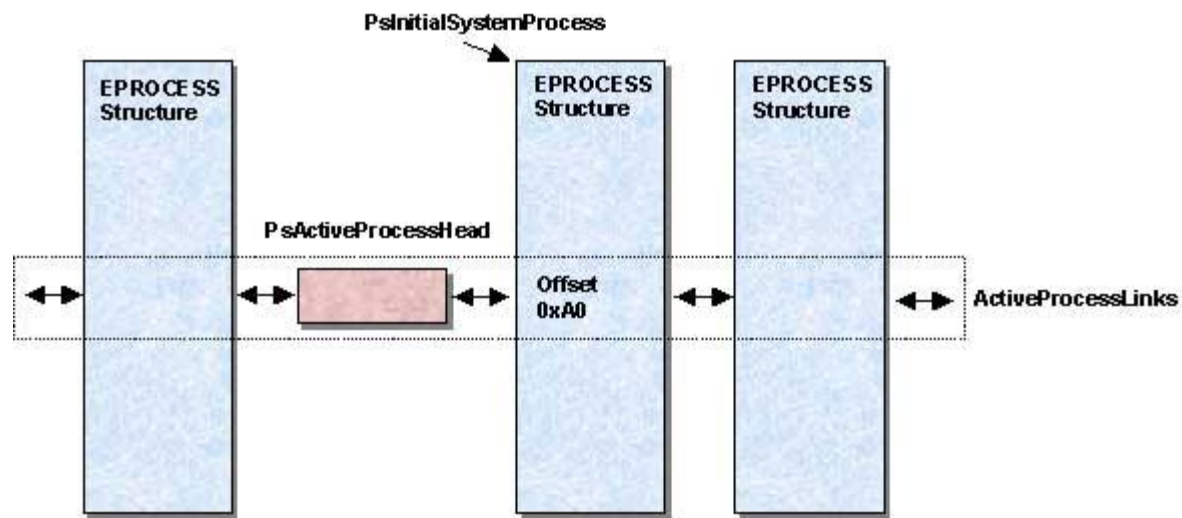
- Blok wykonawczy procesu (executive process block EPROCESS)
- Bloki wykonawcze wątków (executive thread blocks ETHREAD)
- Blok środowiska procesu



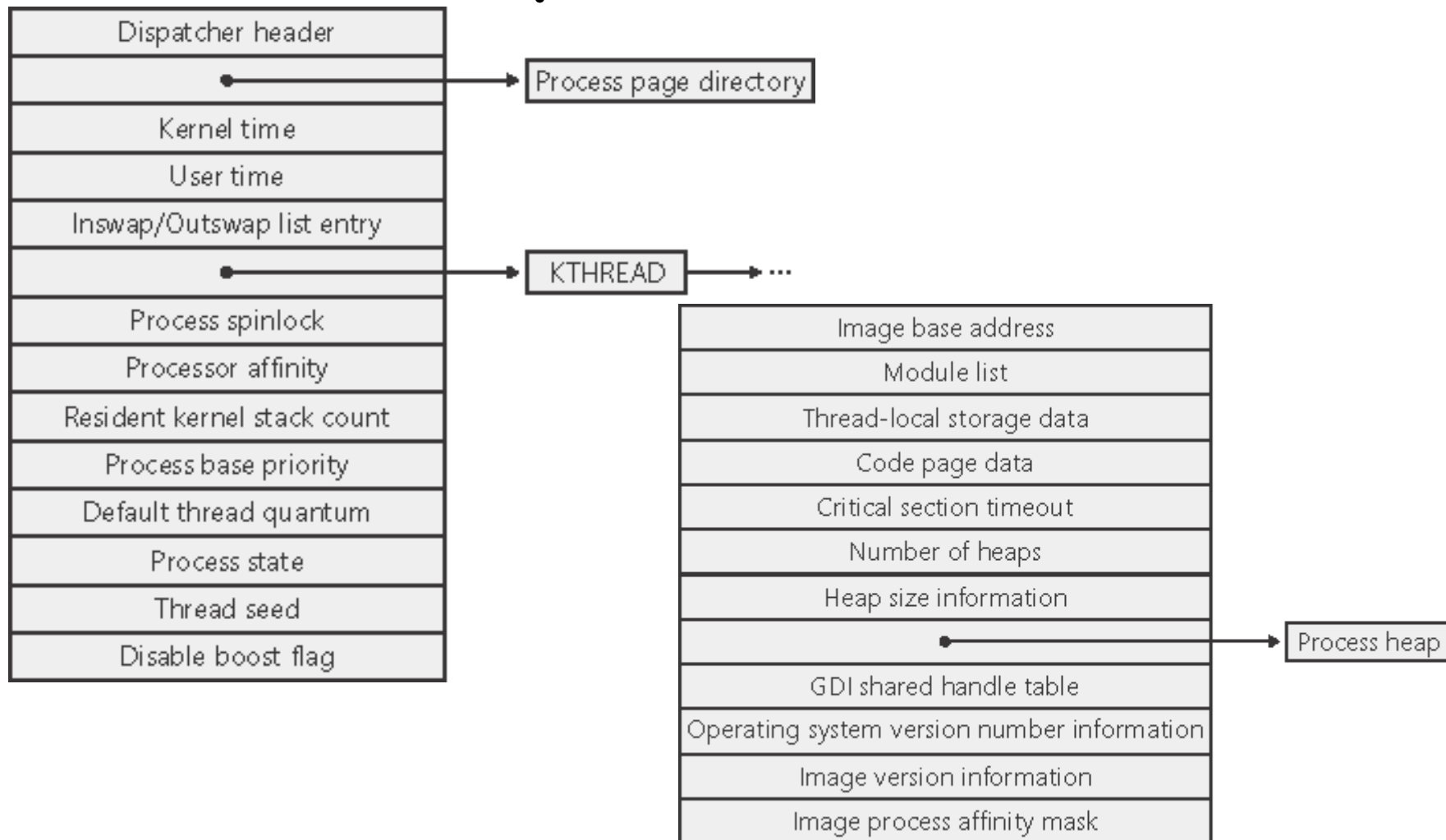
EPROCESS



Kolejka procesów



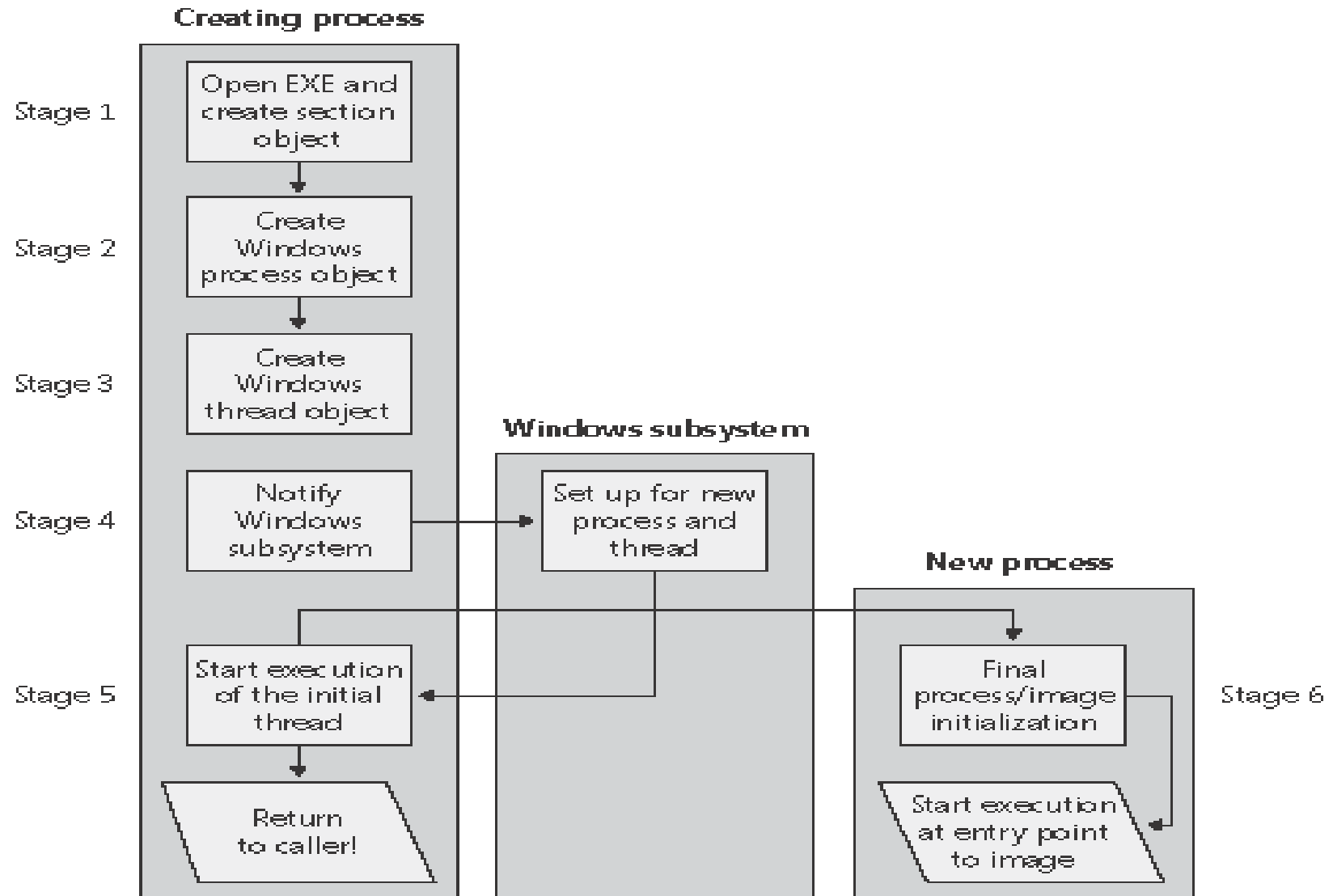
PCB/Kprocess i PEB

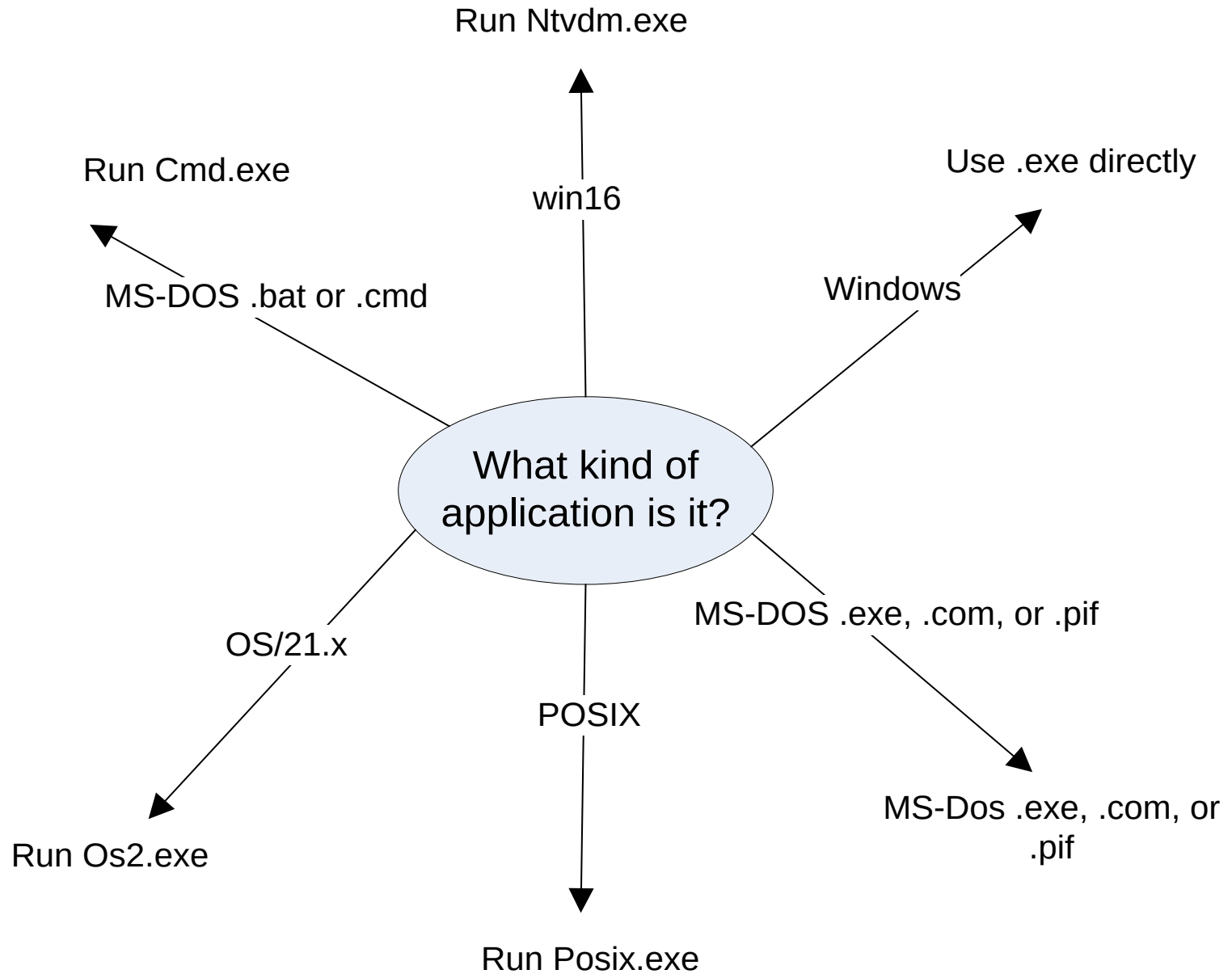


Przeptyw sterowania podczas tworzenia procesu

1. Otwarcie pliku.exe i utworzenie obiektów sekcji
2. Utworzenie obiektów procesu
 1. Ustawienie bloku EPROCESS
 2. Utworzenie inicjalnej przestrzeni adresowej
 3. Utworzenie bloku kernela procesu
 4. Ukończenie tworzenia przestrzeni adresowej
 5. Ustawienie PEB
 6. Ukończenie konfiguracji EPROCESS
3. Utworzenie obiektu wątku
4. Powiadomienie podsystemu (csrss) o nowym procesie
5. Utworzenie wątku głównego
6. Utworzenie kontekstu i uruchomienie wątku

Przeptyw sterowania podczas tworzenia procesu





Stany procesów

- **Nowy** — formowanie procesu, czyli gromadzenie zasobów niezbędnych do rozpoczęcia wykonywania procesu. Po zakończeniu formatowania proces przechodzi w stan *Gotowy*.
- **Wykonywany** — wykonywanie instrukcji programu danego procesu i wynikająca z ich wykonywania zmiana stanu odpowiednich zasobów systemu.
- **Oczekujący** — zatrzymanie wykonywania instrukcji programu danego procesu ze względu na potrzebę przydzielenia dodatkowych zasobów, konieczność otrzymania danych lub osiągnięcia odpowiedniego stanu przez otoczenie procesu (np. urządzenia zewnętrzne lub inne procesy).
- **Gotowy** — oczekiwanie na przydział kwantu czasu procesora (dostępność wszystkich niezbędnych zasobów z wyjątkiem procesora).
- **Zakończony** — zakończenie wykonywania programu, zwolnienie większości zasobów i oczekiwanie na możliwość przekazania informacji o zakończeniu innym procesom lub jądro systemu operacyjnego.

Kończenie procesu

Proces jest wykonywany dopóki nie zajdzie:

- Dowolny wątek wykona funkcję `ExitProcess`
- Dowolny wątek wykona funkcję `TerminateProcess`
- Główny (podstawowy) wątek zwróci sterowanie do systemu (można zapobiec przez wywołanie funkcji `ExitThread`)
- Ostatni wątek procesu zakończy działanie
- Użytkownik zamyka system lub się wylogowuje

- Kiedy proces się kończy, stan obiektu związanego z procesem staje się ustawiony, zwalniając tym samym wątki, które czekały na zakończenie tego procesu.
- Otwarte handlers do plików lub inne zasoby są zamykane automatycznie podczas zamykania procesu.
- Jednakowoż, obiekty same w sobie istnieją dopóki wszystkie otwarte handlers do nich nie zostaną zamknięte.
- Oznacza to, że obiekty mogą pozostać ważne po zamknięciu danego procesu, jeśli inny proces posiada do nich handlers.
- Jeśli proces jest zakończony przez użycie funkcji **ExitProcess**, system wywołuje punkt wejścia każdej podłączonej biblioteki DLL z wartością informującą, że proces odłącza się od DLL-a. Biblioteki DLL nie są informowane, gdy proces jest zakańczany przez użycie funkcji **TerminateProcess**.
- **Ostrzeżenie:** Funkcja **TerminateProcess** powinna być używana tylko w ekstremalnych przypadkach. Użycie jej nie pozwala wątkom na wyczyszczenie zasobów lub zapis danych oraz nie informuje podłączonych DLL-i o zakończeniu procesu.

Wątek

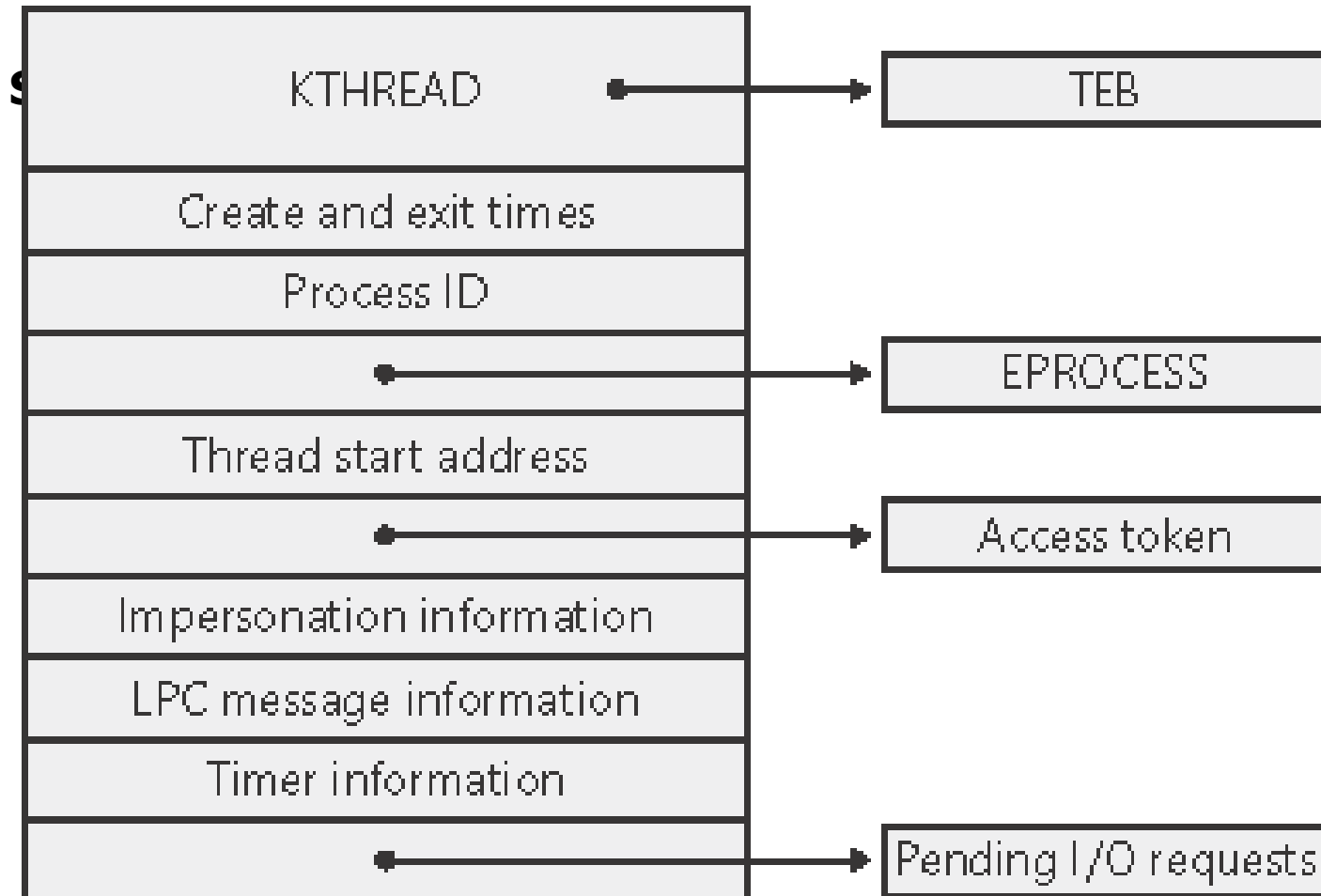
Wątek to jednostka w ramach procesu, jaką Windows szereguje wśród zadań wykonawczych. Wątek musi zawierać:

- Stan rejestrów procesora.
- Dwa stosy: jeden wykorzystywany w kernel mode a jeden w user mode.
- Prywatny obszar przechowywania TLS (thread-local storage) do wykorzystania przez różne podsystemy, biblioteki uruchomieniowe, biblioteki dynamiczne.
- Unikatowy identyfikator wątku.
- Czasem dodatkowo kontekst bezpieczeństwa - dodatkowe dane do wykorzystania w wielowątkowych aplikacjach uruchamianych na serwerach.

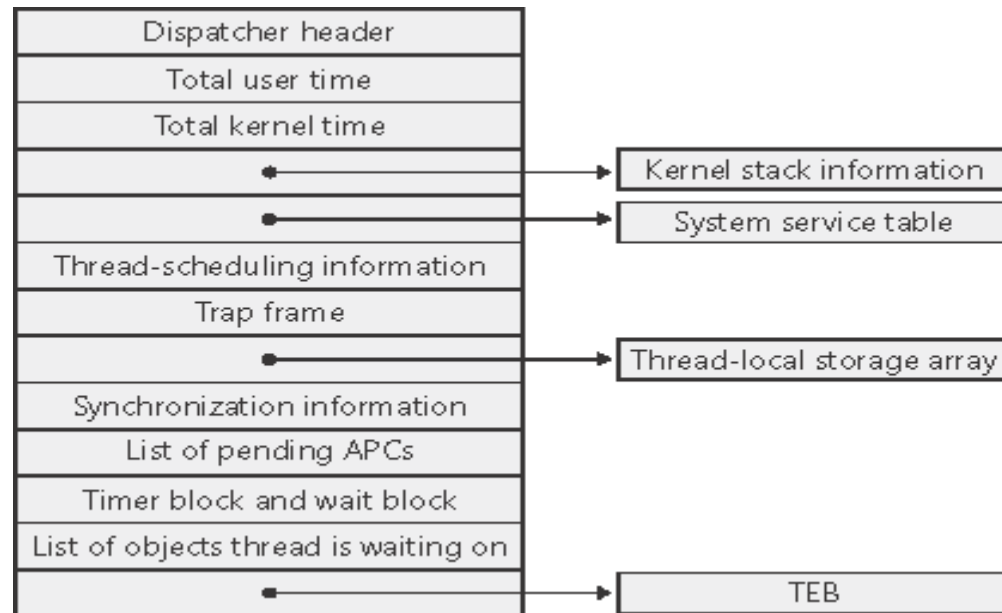
Włókno (Fibre)

- Włókno bywa nazywane lekkim wątkiem i pozwala aplikacji na szeregowanie własnych wątków samodzielnie zamiast zdawać się na globalny dyspozytor.
- Włókna są niewidzialne dla kernela, ponieważ działają wyłącznie w trybie użytkownika. W odróżnieniu od wątku włókno nie zaczyna się od razu wykonywać po utworzeniu - musi być uruchomione.

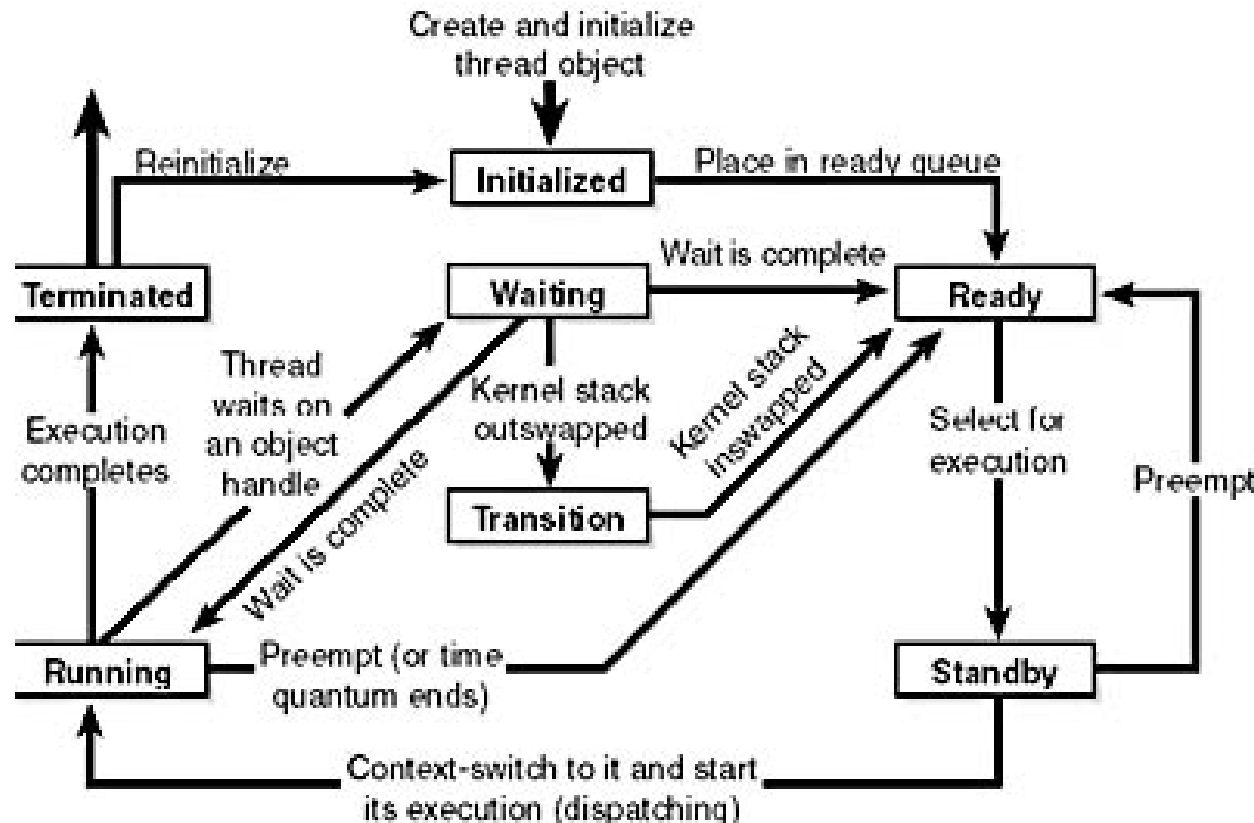
Blok wykonawczy wątku ETHREAD



KTHREAD i TEB



Stany wątku



Stany Wątków

- `THREAD_STATE_INITIALIZED` - inicjalizacja wątku
- `THREAD_STATE_READY` - gotowość oznacza czekanie na wykonywanie. Dyspozytor szuka tylko wątków znajdujących się w tym stanie. Wątek w stanie gotowości z najwyższym priorytetem przechodzi w stan pogotowia.
- `THREAD_STATE_RUNNING` - wykonywany - procesor wykonuje jego instrukcje dopóki jądro go nie wyłączy chcąc uruchomić wątek o wyższym priorytecie, jego kwant upłynął, zakończył swoje działanie lub dobrowolnie zrezygnował z wykonywania się
- `THREAD_STATE_STANDBY` - stan pogotowia. Wątek który znajduje się w tym stanie będzie wykonywany w pierwszej kolejności. W systemie wieloprocesorowym każdy procesor utrzymuje jeden wątek w stanie pogotowia.
- `THREAD_STATE_TERMINATED` - kiedy proces kończy działanie przechodzi w ten stan.
- `THREAD_STATE_WAIT` - wątek jest w stanie oczekiwania jeśli czeka na sygnał od obiektu dyspozytora. Wątek może się znaleźć w tym stanie w momencie gdy: czeka na obiekt synchronizujący jego działanie, operację wejścia-wyjścia. Po otrzymaniu przechodzi w stan gotowości
- `THREAD_STATE_TRANSITION` - stan przejściowy - gdy jądro oczekuje na zasoby niezbędne do działania systemu np. gdy następuje brak pamięci.
- `THREAD_STATE_UNKNOWN`

Tworzenie nowego wątku

1. *CreateThread* tworzy stos dla nowego wątku w przestrzeni adresowej procesu
2. *NtCreateThread* - jest wywoływane i tworzy nowy wątek w stanie zawieszonym.
3. *CreateThread* informuje podsystem win32 o nowym wątku
4. Uchwyt wątku i ID (generowane w punkcie 2) są zwracane ...
5. Jeżeli wątek nie został stworzony jako zawieszony (z ustawioną flagą `CREATE_SUSPENDED`) to wątek zostaje wznowiony

Niszczzenie Wątków

Wątek jest wykonywany dopóki nie wystąpi jedno z poniższych zdarzeń:

- Wątek wywoła funkcję **ExitThread**.
- Inny wątek procesu wywoła funkcję **ExitThread**.
- Funkcja wątku zakończy działanie i zwróci wartość.
- Dowolny wątek wywoła funkcję **TerminateThread** z parametrem w postaci handlera zamykanego wątku.
- Dowolny wątek wywoła funkcję **TerminateProcess** z parametrem w postaci handlera zamykanego procesu (proces również zostaje zniszczony).

Uwagi

Jeśli wątek jest zakończony przez wywołanie funkcji **ExitThread**, system wywołuje punkt wejścia funkcji każdej z powiązanych bibliotek DLL z wartością wskazującą, że wątek jest odłączany od DLLa (chyba że wywołano

funkcję **DisableThreadLibraryCalls**). Jeśli wątek jest zakończony przez wywołanie funkcji **ExitProcess**, punkty wejścia DLLa są wywoływane raz, dla wykrycia że proces jest zwalniany. Biblioteki DLL nie są informowane gdy

wątek jest zakańczany przez wywołanie funkcji **TerminateThread** lub **TerminateProcess**.

Ostrzeżenie: Funkcje **TerminateThread** i **TerminateProcess** powinny być używane tylko w ekstremalnych okolicznościach , ponieważ one nie pozwalają wątkom na wyczyszczenie się (alokacje, handlery itp.), nie informują podłączonych DLL-i i nie zwalniają stosu.

Planowanie zadań

- System Windows XP planuje wątki za pomocą wielopoziomowych kolejek ze sprzężeniem zwrotnym - umożliwia to przemieszczanie się procesów między kolejkami.
- Pomysł polega na rozdzieleniu procesów o różnym zapotrzebowaniu na moc procesora. Jeśli proces zużywa za dużo czasu procesora, to zostanie przeniesiony do kolejki o niższym priorytecie.
- Planista wielopoziomowych kolejek ze sprzężeniem zwrotnym jest określony za pomocą następujących parametrów:
 - liczby kolejek
 - algorytmu planowania dla każdej kolejki
 - metody użytej do decydowania o awansowaniu procesu do kolejki o wyższym priorytecie
 - metody wyznaczającej kolejkę, do której trafia proces potrzebujący obsługi

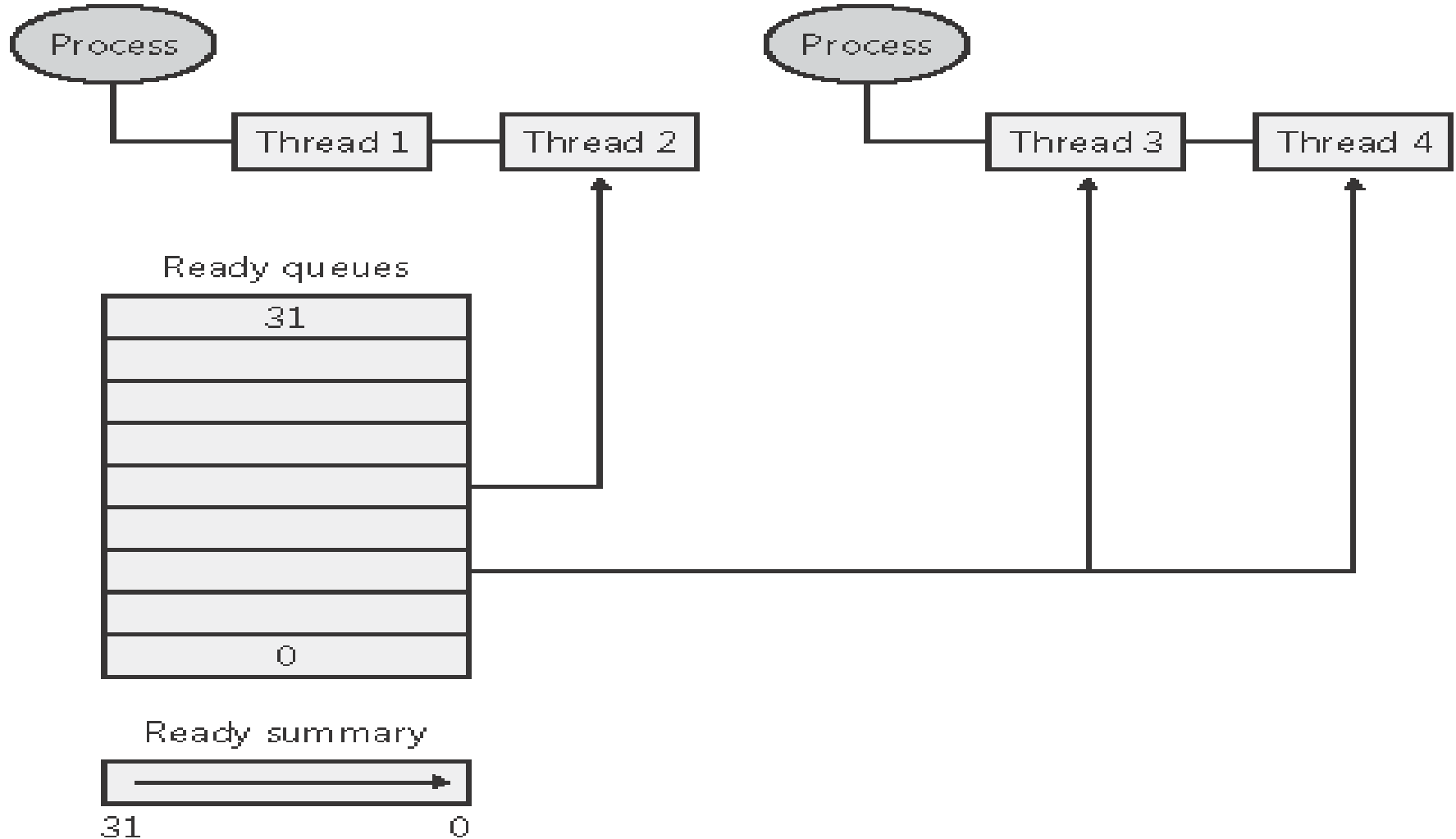
Planowanie zadań (scheduling)

- Windows XP przydziela czas na poziomie wątków (thread granularity)
- Planista (dyzpozytor) bazuje na wyłłaszczaniu w oparciu o priorytety
- Wpierw wykonywane są wątki o najwyższych priorytetach - algorytm round robin z priorytetami.
- Windows XP wykorzystuje 32 (31) poziomy priorytetów System level (0), Variable levels (1-15), Real-time levels (16-31)
- Z punktu widzenia Win32 API
 - Procesy - Idle, Below Normal, Normal, Above Normal, High, Real-time
 - Wątki - Idle, Lowest, Below-Normal, Normal, Above Normal, Highest, Time-Critical

Przydzielanie wątkom czasu procesora

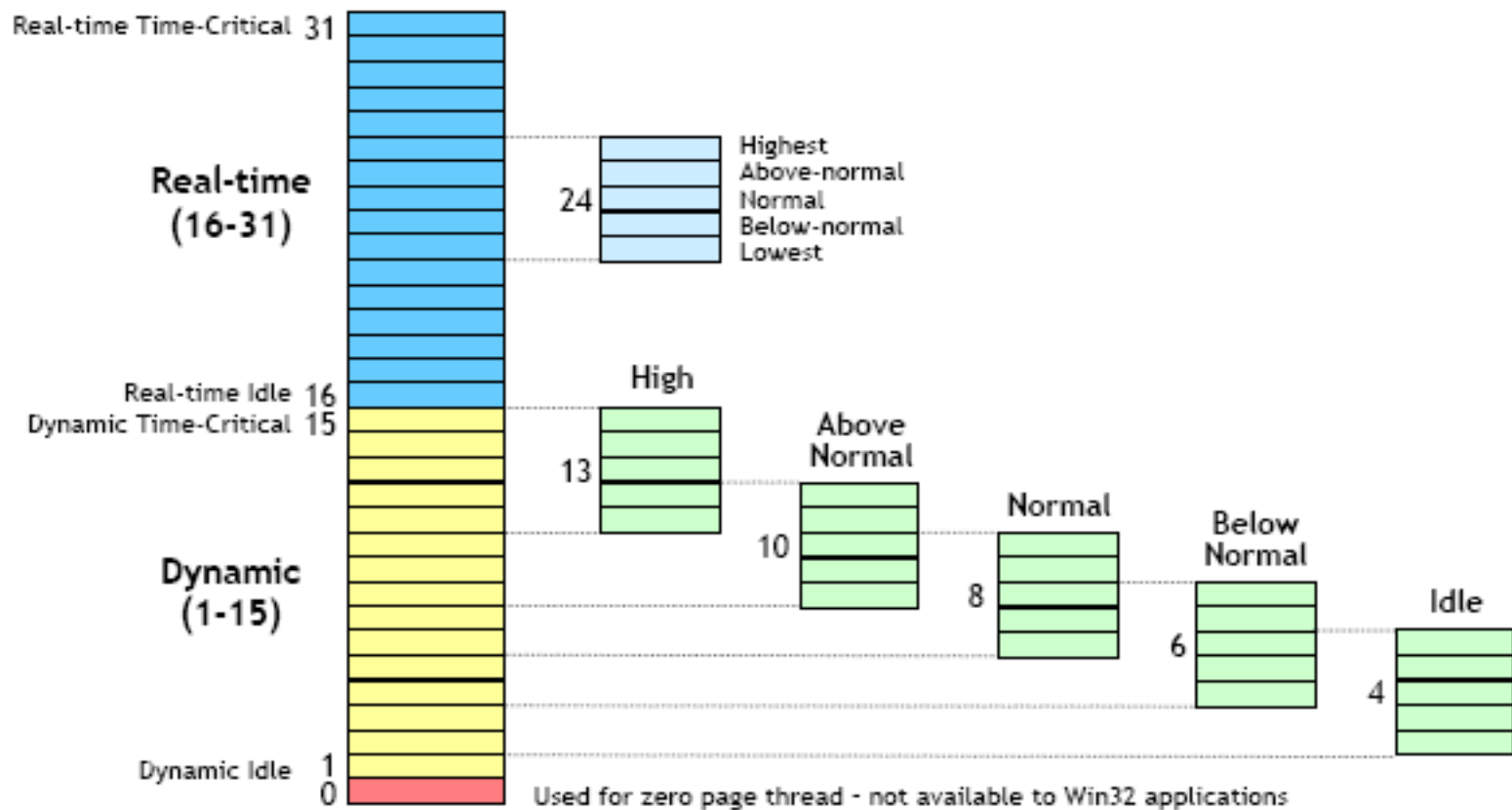
- System przydzielania czasu z wywłaszczeniem opartym na priorytetach (wątek o najwyższym priorytecie działa stale)
- Po upływie kwantu (jednostkowego czasu działania wątku) system sprawdza czy inny wątek o tym samym lub wyższym priorytecie nie czeka na uruchomienie lub czy nie należy obniżyć priorytetu wątku
- Wątek może zostać wywłaszczony przed upływem wyznaczonego czasu przez wątek o wyższym priorytecie.
- Kod przydzielania czasu procesora jest zaimplementowany w różnych modułach jądra. Procedury odpowiedzialne za to zadanie nazywane są wspólnym terminem **dyspozytor**

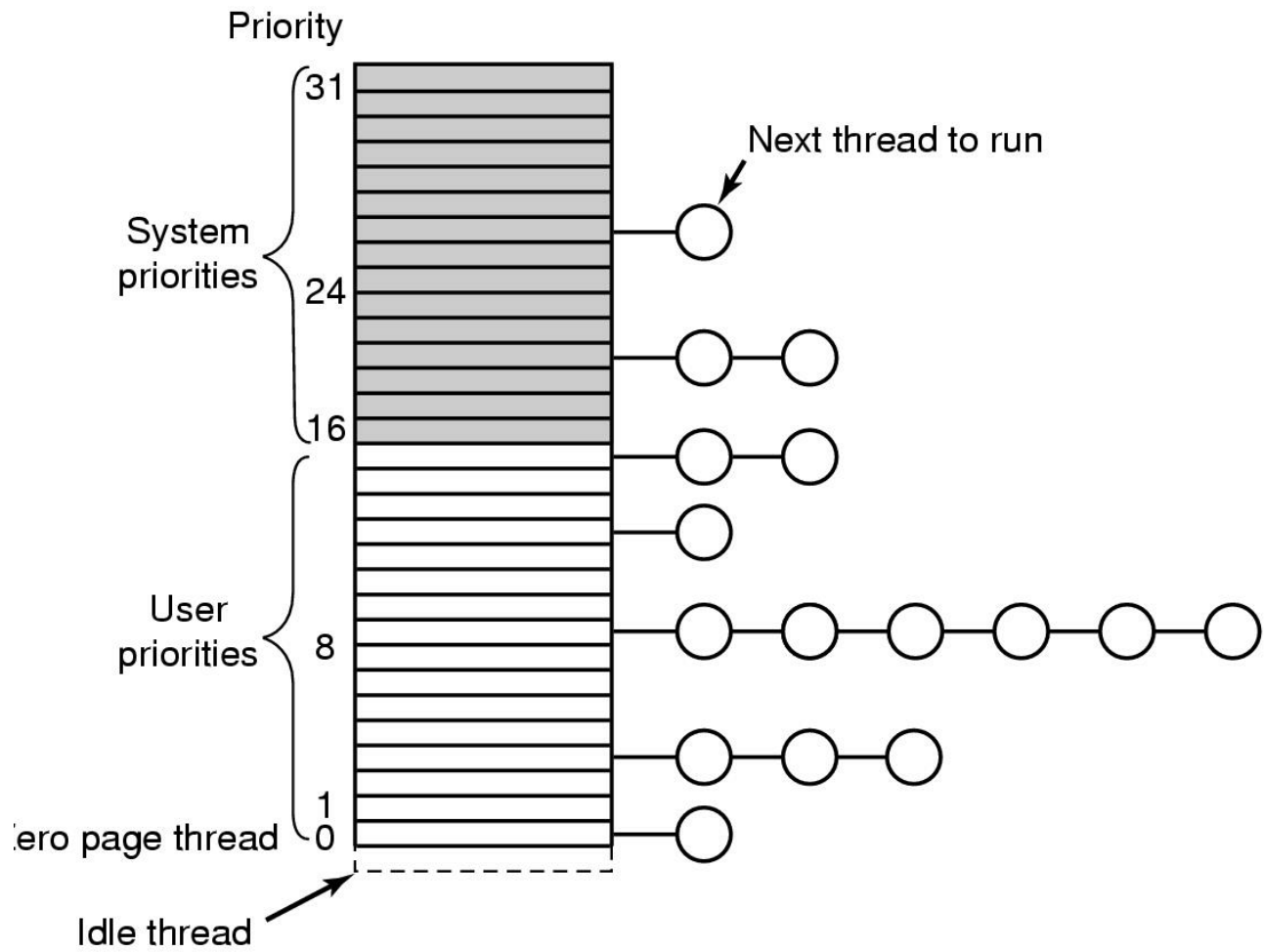
Dyspozytor (Dispatcher Database)



Poziomy priorytetu wątków

- Windows używa 32 poziomów priorytetu wątków (0-31)
 - szesnaście poziomów czasu rzeczywistego (16-31)
 - piętnaście poziomów zmiennych (1-15)
 - jeden poziom systemowy (0) zarezerwowany dla wątku podstawowego
- Nowy wątek dziedziczy priorytet procesu (staje się on jego priorytetem bazowym)
- Wątek ma dwa poziomy priorytetu:
 - bieżący
 - bazowy





Quantum

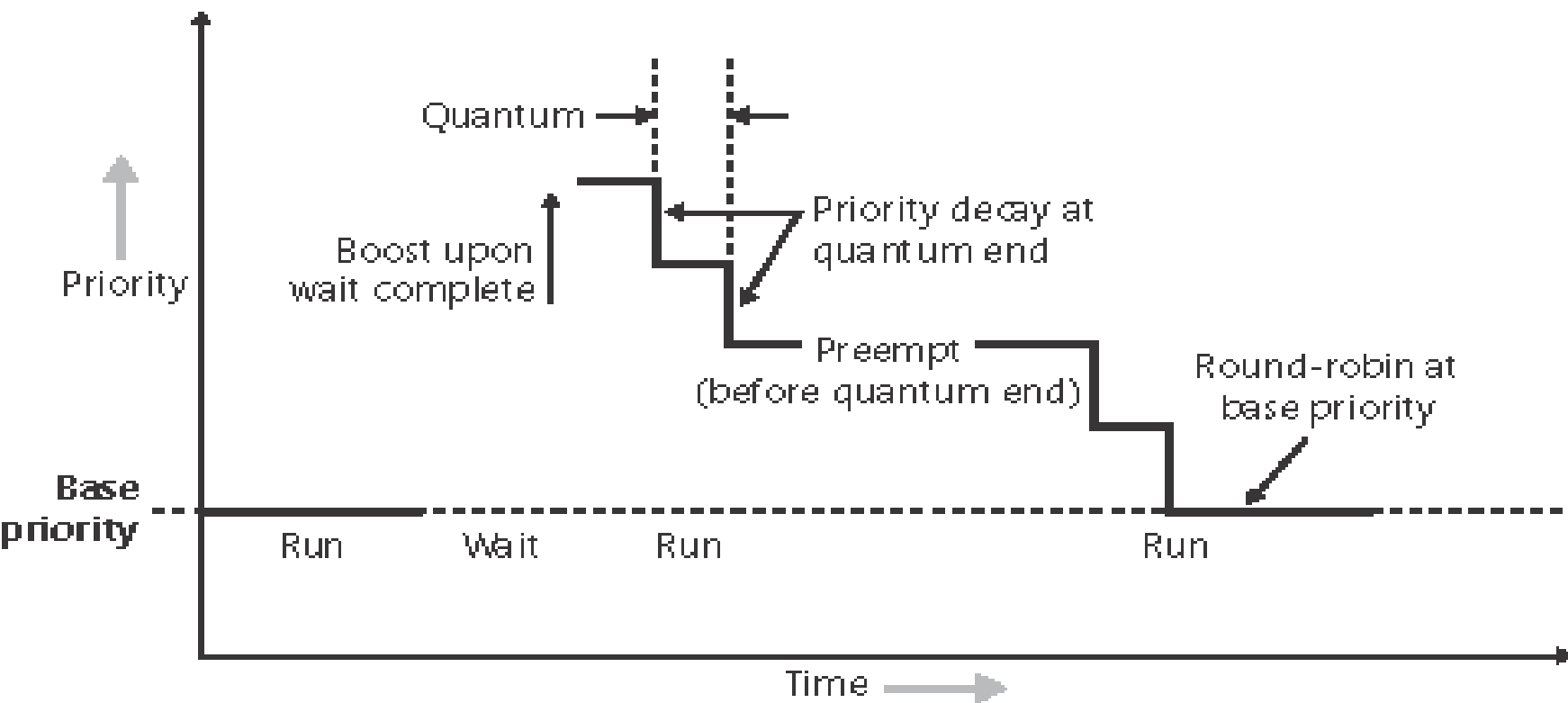
- Kwant czasu jest jednostką czasu przez który działa wątek zanim system sprawdzi czy inny wątek o tym samym priorytecie może być uruchomiony
- W Windows 2000 Professional i Windows XP, wątki działają domyślnie przez 2 interwały czasowe, w Windows Server 12.
- Odmierzanie kwantu
 - Wartości kwantu są trzykrotną wielkością liczby przerw zegarowych (6 lub 12).
 - Pozwala to na pewne rozbieżności tej wielkości w celu lepszego zarządzania czasem (np.. Boosting)

Zwiększanie priorytetu wątku

W pięciu przypadkach Windows może zwiększyć priorytet wątku:

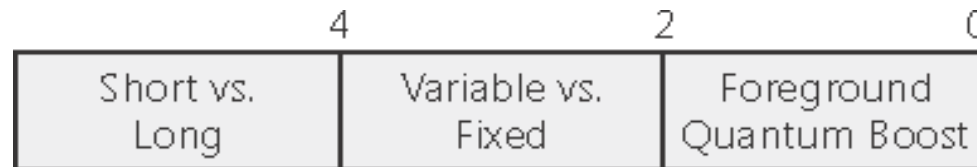
- Po zakończeniu operacji I/O - wartość zwiększenia priorytetu zależy od sterownika (od +1 dla myszki do +8 dla obsługi dźwięku). Co 1 okres zmniejsza się o 1 aż powróci do bazowego
- Po odczekaniu na zdarzenie bądź semafor - priorytet zwiększa się o 1. Po wykorzystanym kwancie powraca do bazowego
- Gdy proces z pierwszego planu zakończył funkcję typu wait - priorytet zwiększony o wartość zmiennej jądra PsPrioritySeparation (0, 1 lub 2)
- Gdy wątki GUI budzą się z powodu aktywności okienek - priorytet zwiększa się o 2
- Proces gotowy do działania nie działał od dłuższego czasu (4s) - priorytet ustawiany na 15 i podwojenie wartości kwantu. Po wykorzystaniu czasu wraca do normy.

W żadnym z tych przypadków wartość procesu nie może przekroczyć 15.



Quantum boosting

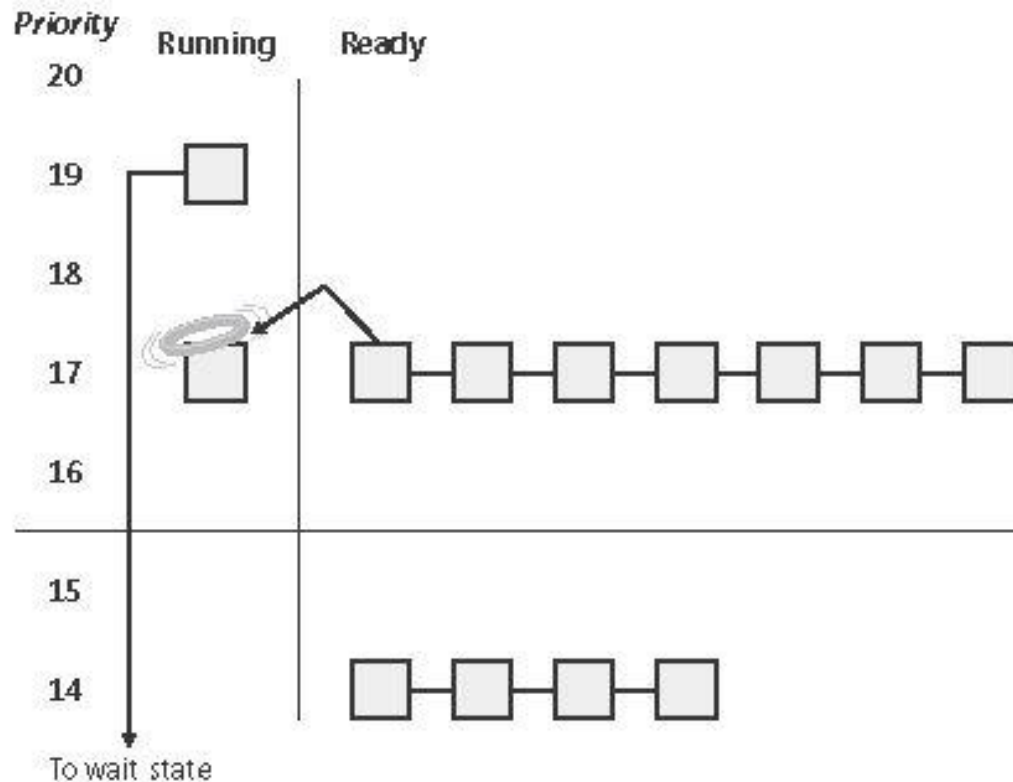
- Quantum Settings Registry Value
 - HKLM\SYSTEM\CurrentControlSet\Control\Priority\Win32PrioritySeparation



Taktyki szeregowania

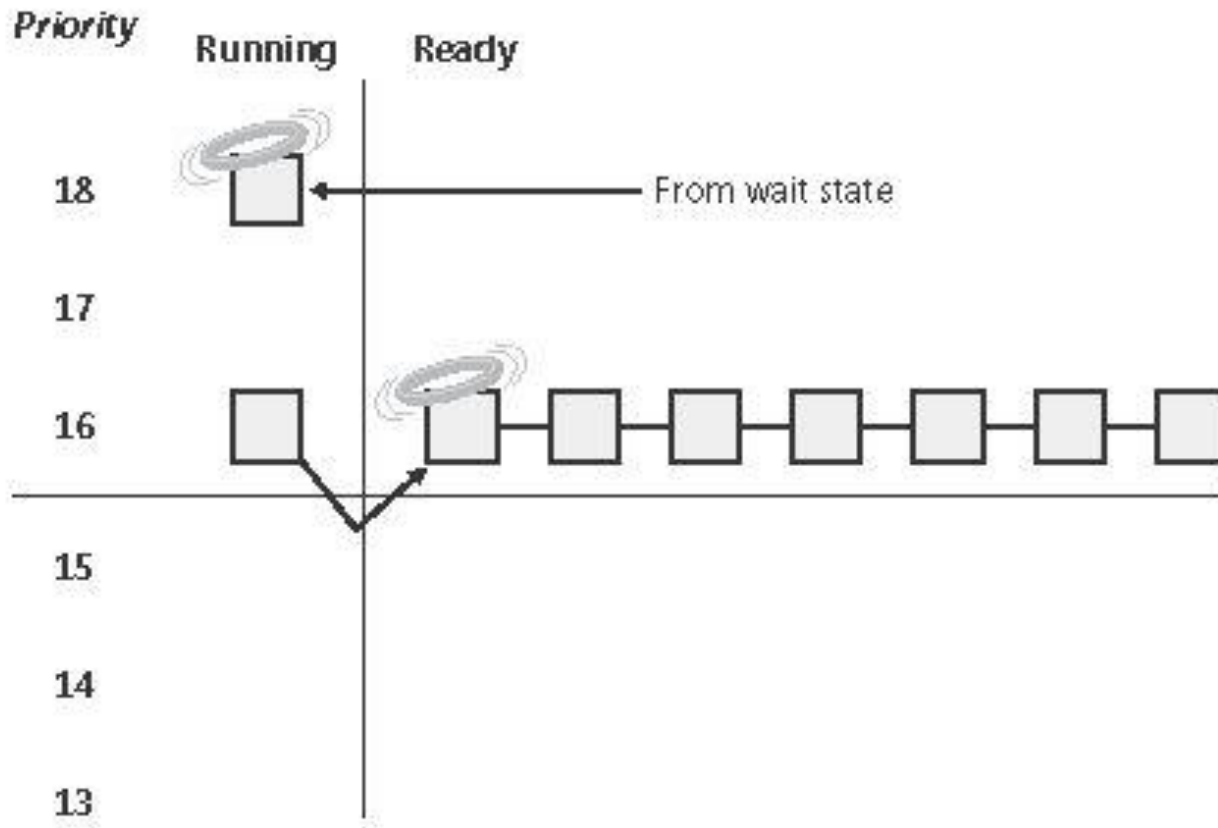
- **Dobrowolne przełączenie** - Wątek może zwolnić dobrowolnie procesor przez przejście w stan oczekiwania na coś dzięki użyciu funkcji typu wait. Kwant czasu dla procesu nie jest resetowany tylko zmniejszany o 1/3 kwantu (tj. 1 jednostkę - pamiętamy, że wewnętrznie kwant składa się z 3 jednostek).

Voluntary switching



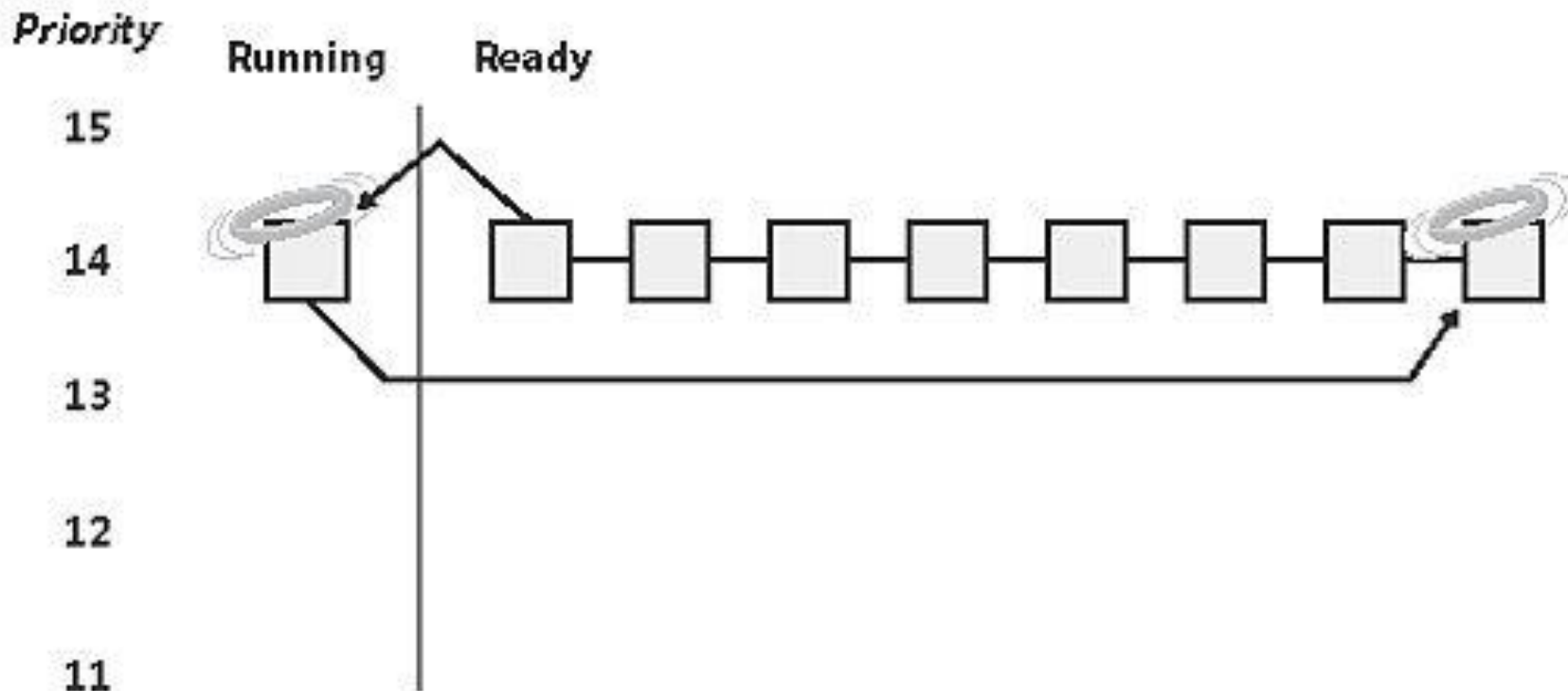
- **Wywłaszczenie** - W tym scenariuszu wątek jest wywłaszczany, gdyż skończyło się oczekiwanie wątku o wyższym priorytecie lub zostały dokonane odpowiednie zmiany w priorytetach wątków. Wywłaszczony wątek jest umieszczany na początku stosownej kolejki:

Preemptive thread scheduling



- **Koniec przydzielonego kwantu** - Gdy wątkowi aktywnemu skończy się kwant, Windows określa czy jego priorytet nie powinien ulec zmniejszeniu i czy inny wątek ma być zaszeregowany do procesora.

Quantum end thread scheduling



- **Zakończenie** - Gdy wątek kończy działanie jest usuwany z kolejki wątków. Jeśli nie ma otwartych dowiązań, to wszystkie obszary pamięci z nim związane są zwalniane.

Podstawowe Funkcje API

CreateProcess – tworzy nowy proces

CreateProcessAsUser – tworzy proces jako użytkownik

OpenProcess – zwraca uchwyt do wybranego procesu

ExitProcess – zamyka proces

TerminateProcess – zabija proces

CreateThread – tworzy nowy wątek

ExitThread – kończy wątek

SuspendThread – zawiesza wątek

TerminateThread – zabija wątek

ConvertFiberToThread konwertuje włókno na wątek

ConvertThreadToFiber – konwertuje wątek na włókno

CreateFiber – tworzy nowe włókno

DeleteFiber – usuwa włókno

Zadania (jobs)

